Fixing inconsistencies between `constexpr` and `consteval` functions

Document Number: P1937R0
Date: 2019-10-06
Author: David Stone (david.stone@uber.com, david@doublewise.net)
Audience: Evolution Working Group (EWG)

A `consteval` function is the same as a `constexpr` function except that:

1. A `consteval` function is guaranteed to be evaluated at compile time.
2. You cannot have a pointer to a `consteval` function except in the body of another `consteval` function.
3. All overrides of a virtual function must be `consteval` if the base function is `consteval`, otherwise they must not be `consteval`.
4. A `consteval` function is evaluated even in contexts that would otherwise be unevaluated.

The first bullet is the intended difference between the two functions. The second bullet is a consequence of the first. The third bullet (`virtual` functions) ends up making sense as a difference between them because of what is fundamentally possible. The final bullet seems like an unfortunate inconsistency in the language. This paper proposes changing the rules surrounding bullet 4 to unify the behavior of `consteval` and `constexpr` functions for C++20. There is also a discussion of `virtual` functions at the end with a proposal for a minor addition targeted at C++23.

Unevaluated contexts

Requiring evaluation of `consteval` functions in unevaluated contexts does not seem necessary. When I asked about this discrepancy on the core reflector, the answer that I got was essentially that evaluation of a `consteval` function may have side-effects on the abstract machine, and we would like to preserve these side-effects. However, `consteval` functions are not special in this ability, so it seems strange to have a rule that makes them special. It seems to me that we should either

1. not have a special case for `consteval`, which would mean that it is not evaluated in an unevaluated operand, or
2. Rethink the concept of "unevaluated operands".

This paper argues in favor of option 1, as option 2 is a breaking change that doesn't seem to bring much benefit.

My understanding is that the status quo means the following is valid:

```
constexpr auto add1(auto lhs, auto rhs) {
    return lhs + rhs;
}
using T = decltype(add1(std::declval<int>(), std::declval<int>()));
```

but this very similar code is ill-formed

```
consteval auto add2(auto lhs, auto rhs) {
    return lhs + rhs;
}
using T = decltype(add2(std::declval<int>(), std::declval<int>()));
```

virtual functions

In C++20, we added support for `constexpr virtual` functions and we also added `consteval` functions. This means that we do not have any backward compatibility concerns binding us. This paper argues that the `consteval` rules are right for `consteval`. It is possible that the `consteval` rules highlight a problem in the rules for `constexpr`, but this paper argues that the current rules are correct.

A `virtual` override is allowed to weaken pre-conditions relative to the function it overrides. With `consteval` vs. an unannotated function, there is in fact no overlap in the pre-conditions. An unannotated function can only be called at run time, but a `consteval` function can only be called at compile time. A `constexpr` function, on the other hand, can be called at either run time or compile time. In other words, a `constexpr` function has weaker pre-conditions than either a `consteval` function or an unannotated function. This suggests that a `constexpr virtual` function can be overridden only by another `constexpr` function (if you have a pointer to a base class with a `constexpr virtual` member function, you should be able to call that at run time or compile time and get polymorphic behavior). However, a `constexpr` function should be allowed to override a non-`constexpr` function (as it would meet all of the pre-conditions of which phase of C++ it can run during).

The alternative approach here is the status quo. If a non-`constexpr` function overrides a `constexpr virtual` function, the compiler still knows that the function that would be called is non-`constexpr` and can thus still gives an error at compile time. This means that there isn't actually a hole opened up in the `virtual` "type system". Overriding a `constexpr` with a non-`constexpr` does not actually cause any problems and has use cases laid out in the paper that originally proposed `constexpr virtual` functions.

This suggests one further change to the rules. A `constexpr` function should also be allowed to override a `consteval virtual` function. However, there is no need for this to be changed for C++20: this is a feature addition, not a bug fix.